Desktop Commander — Complete Functionality Inventory

Compatibility target and implementation checklist for ContextBridge

Desktop Commander is more than a filesystem tool. It provides remote computer access, terminal execution, file editing, document handling, process management, search, device management, and MCP integration.

The tools exposed by the connected Desktop Commander Remote installation were checked and compared with the official documentation. The current connection exposes **31 tools**, including account, diagnostic, and feedback functions.

Below is the functionality inventory to use as a development checklist for ContextBridge.

1. Filesystem operations

Function	Description
`read_file`	Read individual files
`read_multiple_files`	Read several files in one operation
`write_file`	Create, overwrite, or append to files
`edit_block`	Make targeted edits without rewriting an entire file
`create_directory`	Create folders and nested directory structures
`list_directory`	List files and folders recursively
`move_file`	Move or rename files and directories
`get_file_info`	Retrieve size, dates, permissions, type, and other metadata

Additional capabilities include reading selected line ranges, negative offsets to read the ends of files, configurable line limits, exact text replacement, ambiguity detection, and directory access restrictions.

> **Deletion:** The current remote tool inventory does not expose a dedicated delete-file tool. Deletion can be performed through authorized terminal commands. ContextBridge should implement a native deletion function with appropriate safeguards.

2. File and content search

Function	Description
`start_search`	Search file names or file contents
`get_more_search_results`	Retrieve paginated search results
`stop_search`	Cancel an active search
`list_searches`	View active search sessions

Search capabilities include recursive traversal, text-pattern matching, filename searching, large-result pagination, asynchronous searching, and Excel content searching.

3. Terminal and command execution

Function	Description
`start_process`	Start a command or interactive process
`read_process_output`	Retrieve new output from a running process
`interact_with_process`	Send input to an existing process
`force_terminate`	Stop an active terminal session
`list_sessions`	List active terminal sessions

Other supported functionality

- Bash, PowerShell, CMD, and other installed shells.
- Configurable command timeouts.
- Background and long-running execution.
- Output streaming and pagination.
- Persistent REPL sessions.
- Interactive Python, Node.js, and similar runtimes.
- Working with Git, SSH, compilers, package managers, and development tools through terminal commands.
- Capturing stdout, stderr, exit status, and process information.

> **Priority:** This is one of Desktop Commander's most important capabilities to replicate.

4. Process and system management

Function	Description
`list_processes`	List running system processes
`kill_process`	Terminate processes by PID
`list_sessions`	Show managed terminal sessions
`force_terminate`	Terminate a managed session

Also manageable through terminal commands

- Linux systemd services.
- Windows services.
- Installed applications.
- Docker and Podman.
- Git repositories and developer environments.
- Scheduled tasks.
- System logs.
- Network interfaces and diagnostic utilities.

Those are capabilities provided through OS commands, rather than individual dedicated Desktop Commander tools.

5. Document and structured-file operations

These functions are part of the documented Desktop Commander implementation and supported file handlers.

File type	Supported functionality
TXT / Markdown	Read, create, edit, append, search
Python / JS / source code	Read, edit, search, rewrite
JSON / YAML / XML	Read and edit structured text
CSV	Read and analyze, including via terminal tools
Excel (.xlsx, .xls, .xlsm)	Read cells, write data, edit ranges, search sheets
PDF	Extract text, read pages, create and modify PDFs
Word (.docx)	Read, create, inspect, and edit content
Images	Read and present supported image formats
Web URLs	Fetch supported remote file contents

- For PDFs, the remote interface includes a dedicated `write_pdf` tool for creation, insertion, and removal of pages.
- For Excel, it supports sheet selection and cell ranges.
- For DOCX, it supports text-bearing outlines and targeted XML editing.

6. Remote device management

These are the capabilities provided by Desktop Commander Remote.

Function	Description
`list_devices`	Display registered devices and online/offline status
`ping`	Verify device connectivity
`shutdown`	Shut down a remote agent process
`who_am_i`	Identify the authenticated account
Remote tool routing	Execute a tool against a selected registered device
Device pairing	Register and authorize machines
Agent reconnection	Restore remote service availability after interruption

Remote Desktop Commander uses a hosted relay and OAuth-based connectivity. It supports Windows, Linux, and macOS device agents.

> **For ContextBridge:** These capabilities must be self-hosted to avoid relying on Desktop Commander's infrastructure.

7. Configuration and permissions

Function	Description
`get_config`	Retrieve configuration and environment information
`set_config_value`	Change supported configuration values

The configuration system includes

- Allowed filesystem directories.
- Blocked commands.
- Preferred shell.
- File reading and writing limits.
- Telemetry settings.
- Connected-client information.
- Server and operating-system details.

Critical distinction: Filesystem allowlists do not automatically restrict all possible filesystem access through terminal commands. ContextBridge should enforce its security boundaries consistently across both file tools and command execution.

8. Monitoring, diagnostics, and session history

Function	Description
`get_usage_stats`	Retrieve usage metrics
`get_recent_tool_calls`	Inspect recent tool activity
`get_tool_result`	Retrieve pending or completed remote call results
`list_remote_tool_calls`	Inspect remote calls continuing beyond the initial wait
`get_config`	Examine current configuration
`give_feedback_to_desktop_commander`	Open the feedback interface

This covers execution history, troubleshooting, usage visibility, and asynchronous call tracking.

For ContextBridge: Preserve a durable local audit log instead of relying exclusively on in-memory session history.

9. ContextBridge implementation checklist

The following is the actionable scope for building an independent replacement.

Implementation tracker — 0 / 45 complete. Check completed capabilities as they are implemented and verified. All items start unchecked; this is not a live repository status report.

Filesystem	**0/6**	**Verified (date / notes)**
☐ Read and read multiple files		
☐ Create, overwrite, and append files		
☐ Targeted file editing		
☐ Create, move, rename, and delete directories/files		
☐ Recursive listing and metadata		
☐ Path and permission enforcement		

Search	**0/6**	**Verified (date / notes)**
☐ Search filenames		
☐ Search file contents		
☐ Recursive search		
☐ Paginated results		
☐ Asynchronous search sessions		
☐ Cancel active searches		

Terminal	**0/6**	**Verified (date / notes)**
☐ Execute Bash, PowerShell, and CMD		
☐ Persistent interactive shells		
☐ Stream stdout and stderr		
☐ Send input to active processes		
☐ Timeouts and bounded output		
☐ Terminate process trees		

System and processes	**0/4**	**Verified (date / notes)**
☐ List and terminate processes		
☐ Manage authorized system services		
☐ Inspect system information and logs		
☐ Support development tools and Git		

Documents	0/5	Verified (date / notes)
☐ Excel read/write/edit/search		
☐ PDF read/create/modify		
☐ Word read/create/edit		
☐ Images and supported URLs		
☐ CSV and structured data handling		

Remote access	0/6	Verified (date / notes)
☐ Linux and Windows native agents		
☐ Device enrollment and revocation		
☐ Online/offline status and heartbeat		
☐ Encrypted bidirectional connection		
☐ Self-hosted relay with reconnect		
☐ Per-device command routing		

Security and operations	0/6	Verified (date / notes)
☐ OAuth-compatible MCP client access		
☐ Per-device and per-tool permission profiles		
☐ Credential and secret isolation		
☐ Complete audit and execution history		
☐ Configurable scopes and resource limits		
☐ Recovery after interrupted operations		

Synchronization (ContextBridge addition)	0/6	Verified (date / notes)
☐ Bidirectional file transfer		
☐ Directory synchronization		
☐ Checksums and change detection		
☐ Conflict preservation		
☐ Resumable transfers		
☐ Recoverable deletion propagation		

10. What Desktop Commander does not provide as a dedicated synchronization system

For this objective, there is an important difference between remote access and automatic synchronization.

Desktop Commander already lets an AI assistant read a file from **Friday**, modify it, and write a file to **Thanatos**. However, its documented toolset is not a continuous bidirectional file synchronization engine with conflict resolution.

ContextBridge could go beyond parity by providing native synchronization, resumable transfers, hash verification, explicit conflict handling, and recovery after network disconnections.

> **Recommendation:** Use Desktop Commander's current tools as the compatibility target, then add ContextBridge's independent device infrastructure and bidirectional synchronization.

> **Acceptance test:** Desktop Commander can be completely disabled while ContextBridge continues operating against Friday and Thanatos using its own agents, permissions, transport, and tools.